



Beyond element-level understanding: Explicit relational understanding for GUI agents

Longhui Ma^a, Di Zhao^a, Siwei Wang^{b,*}, Zhao Lv^b, Miao Wang^b

^a College of Computer Science and Technology, National University of Defense Technology, Changsha, China

^b The Intelligent Game and Decision Lab, Academy of Military Sciences, Beijing, China

ARTICLE INFO

Keywords:

Graphical user interfaces
Multi-modal large language models
GUI agents
Human-computer interaction

ABSTRACT

With the emergence of Multi-modal Large Language Models (MLLMs), GUI agents have become increasingly important as intelligent assistants capable of understanding and interacting with graphical user interfaces (GUIs). Unlike natural images, GUI elements are artificially designed and intricately organized, exhibiting complex relationships to one another. However, existing GUI agents largely overlook the comprehension of these inter-element relationships, which are crucial for accurately interpreting element roles and functional dependencies within complex interfaces. This limitation significantly hinders agents' generalization capabilities, particularly on tasks requiring deep relational understanding. To address this gap, we propose RelationAgent, an advanced framework that explicitly incorporates relational understanding as a core learning objective. To enable this capability, we introduce RelationGUI, the first large-scale, cross-platform dataset featuring fine-grained annotations of spatial, functional, and hierarchical relationships among GUI elements. To the best of our knowledge, current GUI datasets and benchmarks fail to comprehensively capture and evaluate this critical capability, making RelationGUI the first dataset specifically designed to address this gap. Experimental results across multiple benchmarks demonstrate that RelationAgent significantly enhances performance in grounding and navigation tasks. Most importantly, evaluation on RelationQA, a specialized QA benchmark derived from RelationGUI, highlights the severe limitations of existing GUI agents in the relational reasoning task (below 30% accuracy). In contrast, RelationAgent achieves an accuracy of 49.7%, underscoring both the critical importance and the inherent challenges associated with relational understanding in GUI environments.

1. Introduction

Graphical User Interfaces (GUIs) represent an essential component of contemporary digital systems, enabling users to interact visually with software through GUI elements [1,2]. Recent advances in Multi-modal Large Language Models (MLLMs) [3,4] have driven increased research interest toward the creation of intelligent GUI agents [5,6] capable of effectively understanding and interacting within GUI environments. These agents offer significant potential for automating software tasks, supporting accessibility requirements, and enabling generalization across diverse platforms, thereby opening new avenues for advancements in human-computer interaction [7].

Current studies on GUI agents primarily emphasize two core abilities: grounding-mapping natural language to GUI elements [8], and navigation-executing multi-step tasks [9,10]. To support these abilities, most methods predominantly adopt techniques originally developed for natural image processing when analyzing GUI images. However, this adaptation overlooks an important distinction: unlike natural images,

where objects typically exhibit arbitrary spatial distributions and weak inter-object semantics, GUI elements are systematically designed and strategically organized to convey specific semantic meanings and functional hierarchies [2]. Ignoring this characteristic limits the agent's ability to reason about element relationships, which is critical for accurate grounding and task execution on tasks requiring deep relational understanding.

For example, in Fig. 1, executing the task “Open the alarm of 10:00” highly depends on the functional relationship between the text label “10:00” and the toggle which controls this alarm. Otherwise, the toggle merely conveys low-level functional semantics, such as “a toggle is off” and results in an incorrect prediction answer [5,9]. While mobile and web platforms in GUI agents can provide rich structured information (e.g., HTML/XML), existing datasets and benchmarks in this domain mainly focus on extracting and filtering element-level attributes. They largely overlook the relational information inherently present in the structural organization of these interfaces [11,12]. Similarly, purely visual approaches without structural information on desktop platforms

* Corresponding author.

E-mail address: wangsiwei13@nudt.edu.cn (S. Wang).

<https://doi.org/10.1016/j.patcog.2026.113262>

Received 9 October 2025; Received in revised form 18 January 2026; Accepted 7 February 2026

Available online 10 February 2026

0031-3203/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

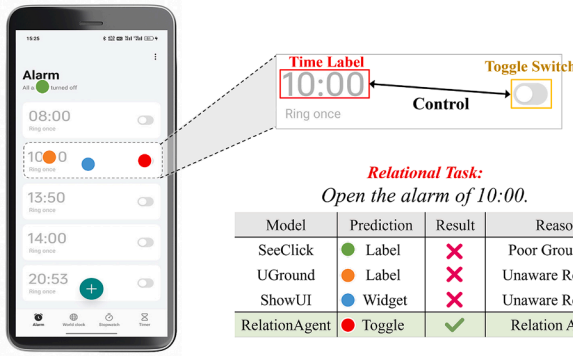


Fig. 1. Illustration of functional relationships in GUI and prediction comparison among agents on a relational task. RelationAgent correctly identifies the relationship thanks to its relational understanding capability, while other agents fail to capture the functional dependence.

mainly center on element identification (e.g., text, icons), but seldom address how these elements interact [13]. Some studies explored relationships between GUI elements, but their focus remains largely limited to spatial dependencies, without modeling functional relationships that are essential for deeper relational understanding [14]. This lack of attention to relationships among GUI elements leads to agents struggle with relational understanding, ultimately limiting their ability to generalize on relational tasks, as illustrated in Fig. 1.

To address this limitation, we argue that the key is to introduce and explicitly model the deep functional relationships among GUI elements [15]. We introduce RelationAgent, which employs a three-stage training paradigm: Grounding-Relation-Navigation, explicitly incorporates relational understanding as a core intermediate learning objective. Our method first grounds the interface by perceiving visual elements, then constructs an internal representation by modeling structural relationships among these elements, and finally performs goal-directed navigation based on this relational understanding. To support the core capability of RelationAgent, we construct RelationGUI, the first large-scale, cross-platform dataset with fine-grained annotations of relationships among GUI elements. Compared to existing datasets for GUI agents, RelationGUI not only provides information about GUI elements but also, more importantly, offers positional, functional, and hierarchical relationships among these elements, thereby providing a valuable resource for training agents to develop relational understanding. Our experimental results demonstrate that RelationAgent not only achieves remarkable performance on grounding and navigation benchmarks, but also reveals the inherent limitations of existing GUI agents in relational reasoning on RelationQA, a dedicated benchmark designed to quantify relational reasoning capabilities of GUI agents. Our main contributions can be summarized as follows:

1. We construct RelationGUI, the first dataset featuring detailed relational annotations among GUI elements, accompanied by RelationQA, a benchmark explicitly designed for evaluating relational reasoning capabilities of GUI agents.
2. We propose RelationAgent, a framework that extends the conventional two-stage paradigm to three-stage Grounding-Relation-Navigation training pipeline, bridging gaps between grounding and navigation.
3. Experimental results demonstrating that RelationAgent performs well on the grounding and navigation benchmarks. And its superior results on the RelationQA benchmark further underscore the importance of relational understanding in GUI agents.

2. Related work

2.1. GUI agents

Building on recent advances in Large Language Models (LLMs) and multi-modal integration, powerful MLLMs such as GPT-4V [16], Gemini-Pro [17], and open-source models like the Qwen-VL series [4,18] have demonstrated strong performance across a wide range of vision-language tasks. In the context of GUI understanding [19], MLLMs provide a foundation for GUI agents. GUI agents aim to understand users' instructions and interact with the GUI to complete tasks. Early work has primarily focused on web and mobile platforms with structured data [12,20], while recent approaches have extended to multi-platform and purely visual settings [5,21], aiming to improve generalization across diverse GUI environments.

2.2. Pattern recognition and GUI agents

Pattern recognition focuses on learning structured regularities and relationships from data to support tasks such as classification, inference, and decision making [22,23]. Beyond recognizing isolated features, modern pattern recognition increasingly emphasizes structured and relational representations that capture dependencies among entities [24].

GUI agents can be viewed as a structured instantiation of pattern recognition, where the objective extends from identifying individual interface elements to understanding their spatial layouts, hierarchical organization, and functional interactions [12,25]. Compared to natural images, GUI environments exhibit stronger structural constraints and recurrent design patterns, making relational modeling a central requirement for effective perception and interaction [26].

From a pattern recognition perspective, relational understanding in GUI agents corresponds to structured pattern modeling over detected GUI elements. Explicitly modeling such relations provides a principled bridge between low-level perception and high-level action planning, and aligns GUI agent research with broader advances in relational and structured pattern recognition [27].

2.3. Datasets and benchmarks for GUI agents

Early progress in GUI agent research has been driven by the availability of structured datasets across mobile and web platforms. Building on foundational efforts of Rico [34], the first large-scale mobile GUI video dataset, and AitW [30], which includes over 700k human-collected episodes, researchers have extensively explored GUI understanding in mobile and web environments [9,20]. Recent efforts have begun to extend to new settings, including more platforms like desktop, XR [32,33] and complex software interaction traces [35], enabling broader generalization across platforms and use cases.

Despite these advances, most current GUI datasets and benchmarks regard element-level information and step-level trace as primary annotations, overlooking the relationships among GUI elements, as illustrated in Table 1. This limitation makes it challenging for models to effectively learn and utilize the underlying relationships directly from the data. Understanding and explicitly modeling the relationships among GUI elements remains an under-explored yet crucial direction. Our RelationGUI dataset directly addresses this by providing explicit relational annotations.

3. RelationGUI: A cross-platform dataset for modeling GUI elements relationships

Although existing GUI datasets have made contributions in terms of diversity, authenticity, and task complexity [12,33], they mainly focus on element-level information and step-level trace. Typically, they lack

Table 1

Comparison of RelationGUI to existing GUI datasets. We consider dataset size (as number of task demonstrations), diversity (as number of unique task instructions and apps/websites), device platform (web, mobile and desktop), and the main tasks dataset intends to apply (grounding task, relational task and navigation task). ‘/’ means that paper doesn’t provide precise statistics.

Dataset	Human demos	Unique instr.	Apps or websites	Env Type			Main Tasks		
				Web	Mobile	Desktop	Grounding	Relation	Navigation
MiniWeb + + [28]	17,971	100	114	✓	✗	✗	✓	✗	✓
WebShop [29]	1566	1566	1	✓	✗	✗	✓	✗	✓
Mind2Web [12]	2350	2350	137	✓	✗	✗	✓	✗	✓
AITW [30]	715,142	30,378	357	✗	✓	✗	✓	✗	✓
AndroidControl [11]	15,283	14,548	833	✗	✓	✗	✓	✗	✓
OmniAct [31]	9802	9802	200	✓	✗	✓	✓	✗	✓
OSWorld [32]	369	369	10	✓	✗	✓	✓	✗	✓
GUI-World [33]	24,758	12,379	/	✓	✓	✓	✓	✗	✓
RelationGUI	37,724	37,724	963	✓	✓	✓	✓	✓	✗

explicit annotations of relationships among GUI elements, which limits their ability to support or evaluate agents’ relational understanding capabilities. To address this gap, we present RelationGUI, a large-scale, cross-platform dataset that provides fine-grained relational annotations among GUI elements. The construction of RelationGUI follows a structured annotation framework and results in 37k high-quality samples of element relationships. It is specifically designed to support training and evaluation of relational understanding in GUI agents.

3.1. Relationship annotation method

Unlike prior GUI datasets that primarily focus on element-level attributes, RelationGUI introduces structured annotations of inter-element relationships, formulated as triplets $r = (e_i, \rho_{ij}, e_j)$, where e_i and e_j denote GUI elements, and ρ_{ij} refers to the specific relationship between them. We categorize the ρ_{ij} into three complementary dimensions: spatial, functional, and hierarchical.

- **Spatial Relationships** $\rho_{ij}^{\text{spatial}}$: Describe the relative position between GUI elements. For element pairs, we annotate spatial relationships into three categories: *left/right*, *top/bottom*, and *others*. For groups of three or more elements, spatial relationships are classified as *horizontal*, *vertical*, and *others*.
- **Functional Relationships** ρ_{ij}^{func} : Capture logical or operational dependencies between elements. Inspired by principles from Norman’s cognitive theory of interaction design [36], we define four types: **trigger**, **complement**, **parallel**, and **none**.
 - **Trigger**: Refers to causal relationships where one element directly affects the behavior of another (e.g., a dropdown menu that controls font size in Microsoft Office).
 - **Complement**: Reflects contextual or auxiliary support, where one element enhances the interpretability of another (e.g., an icon accompanied by a descriptive label).
 - **Parallel**: Describes elements that operate independently but are grouped logically or spatially (e.g., alignment buttons in a formatting toolbar).
 - **None**: Indicates that no functional relationship exists between the elements.
- **Hierarchical Relationships** $\rho_{ij}^{\text{hierar}}$: Represent structural nesting patterns in GUI layouts, such as DOM tree paths or containment hierarchies in software interfaces.

Although spatial and hierarchical relations both encode positional information, they differ in semantic scope and representational granularity. Spatial relations characterize geometric proximity and directional layout between GUI elements (e.g., A is to the left of B, C is above D), capturing the visual organization that underlies perceptual grounding [37]. Hierarchical relations, in contrast, describe structural containment and interface-level dependencies (e.g., a button belongs to a toolbar, a menu contains items), reflecting parent-child organization specified by GUI

frameworks rather than purely spatial adjacency [38]. Functional relations complement these structural relations by modeling interactive or operational dependencies between elements (e.g., a checkbox controls an input field), which are essential for reasoning about causal behaviors in GUI interactions [14].

3.2. Platform-aware data collection

Given the diversity of GUI environments, we adopt platform-aware strategies for data collection and annotation. Specifically, web data are collected through automated crawling followed by rule-based filtering. Mobile data are reorganized and relabeled based on existing public datasets. Desktop data are manually curated and annotated from a set of representative software applications. The overall dataset collection pipeline is illustrated in Fig. 2.

3.2.1. Web-based data subset construction

To construct the web subset, we curated a corpus of 2480 top-ranked websites across 33 diverse categories (see Appendix). For each site, we used Selenium to extract DOM structures and then filtered sibling nodes sharing the same parent as input for the following stages, which aligns with interface design principles indicating that meaningful relationships often occur within structural groups [2]. To identify high-quality relational candidates, we employed a three-stage filtering pipeline as illustrated in Fig. 2. First, element pairs were heuristically scored based on geometric features (e.g., position, size, alignment). Next, MLLMs [16] performed semantic filtering to discard noisy pairs. Finally, annotators manually verified the remaining pairs to ensure contextual accuracy. Filtered pairs were merged into clusters via a union-set strategy. For each cluster, functional relationships (ρ_{ij}^{func}) were generated using MLLMs, while spatial ($\rho_{ij}^{\text{spatial}}$) and hierarchical ($\rho_{ij}^{\text{hierar}}$) relationships were determined via rule-based analysis. The resulting web subset comprises 10,586 annotated relationships across 917 pages.

3.2.2. Mobile-based data subset construction

For the mobile platform, we leverage the MobileViews dataset [39], which contains over 600,000 screenshot-view hierarchy pairs from more than 20,000 Android applications. Each sample offers detailed control-level information, including bounding boxes, XML attributes, descriptions, and metadata, making it suitable for relational analysis. Following a similar pipeline as illustrated in the web-based subset, we apply a three-stage filtering process to identify relational candidates. A key distinction lies in the source of hierarchical annotations: while web data rely on HTML paths, mobile data use control paths derived from the Android view hierarchy. The resulting mobile subset comprises 20,284 annotated relationships across 7210 unique interfaces.

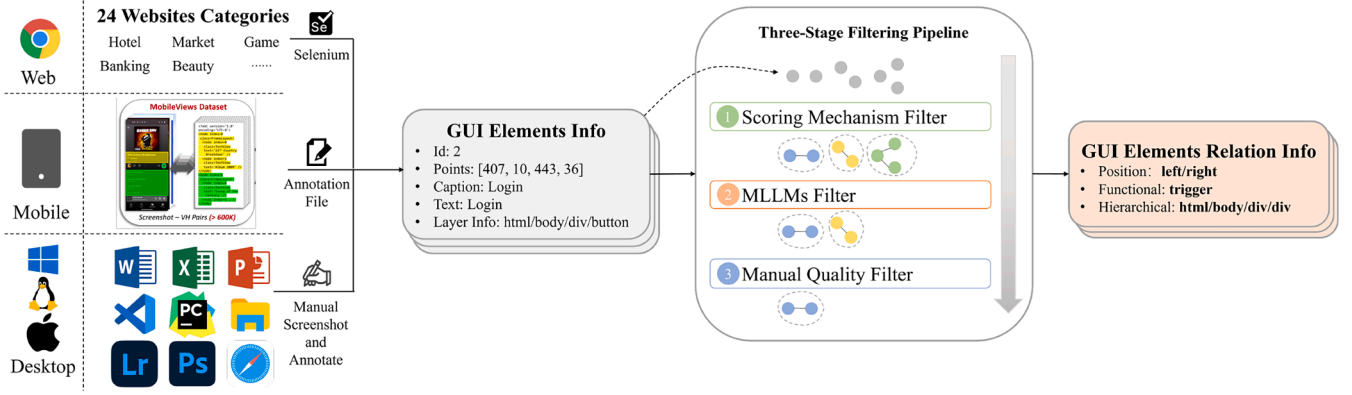


Fig. 2. An overview construction pipeline of RelationGUI. We adopt platform-specific approaches for web, mobile, and desktop environments. The ultimate objective is to extract both GUI element information and their associated relational information.

3.2.3. Desktop-based data subset construction

Due to the lack of comprehensive open-source datasets tailored for relational understanding on desktop platforms, we construct a desktop subset via a manual data collection and annotation workflow. We select 46 representative desktop applications (18 on Windows, 21 on Ubuntu, and 7 on macOS) spanning various domains such as productivity and multimedia. For each application, we capture screenshots during typical user interactions to reflect realistic usage contexts. Annotations are performed using LabelMe, with a focus on groups of GUI elements that exhibit functional relationships, rather than annotating all visible elements. Element-level metadata (e.g., bounding-box, icon captions), functional and hierarchical relationships (manually assigned), and spatial relationships (inferred via rule-based logic) are annotated. To increase diversity and support robust model training, we include randomly selected negative samples consisting of element pairs without meaningful functional relationships. All annotations are manually verified to ensure semantic consistency and remove noise. The final desktop subset comprises 6854 annotated relation samples.

3.2.4. Annotation quality control

Lastly, all task demonstrations are manually verified by the authors to ensure data quality and consistency. First, all collected samples are formatted into a unified JSON structure, as illustrated in Fig. 3. Second, all bounding boxes are strictly aligned with visually clear icons or texts, avoiding loosely-defined or semantically ambiguous regions. Third, we remove samples that are incorrect or of low quality. By integrating annotated samples across web, mobile, and desktop platforms, we construct a unified, cross-platform dataset comprising 37,724 relational samples. The annotated example is illustrated in Fig. 3. The data collection pipeline is effective and yields high-quality data. Details on the filtering algorithm, dataset statistics, and MLLMs prompting templates are provided in the Appendix.

3.3. RelationQA: A benchmark for evaluating relational reasoning

Evaluating an agent's understanding of relationships among GUI elements in isolation is both critical and challenging. To address this, we introduce RelationQA, a targeted benchmark derived from RelationGUI to assess whether an agent can correctly identify functional associations between pairs or groups of GUI elements. Candidate instances are randomly sampled from RelationGUI and filtered with GPT-4o [16] to ensure clarity and relevance, resulting in 1009 question-answer pairs. To isolate relational reasoning from grounding accuracy, we adopt the Set-of-Mark (SoM) strategy [40], which explicitly marks the GUI elements involved in each query. Specifically, each target element is enclosed by a red bounding box corresponding to its annotated bounding box, as illustrated in the middle panel of Fig. 4. Additionally, to address challenges caused by low-resolution components, particularly on desktop

interfaces, we provide cropped views of target elements as additional input, enabling the agent to focus on relevant content and suppress visual noise. This design encourages the agent to attend to task-relevant regions while suppressing irrelevant visual context. The cropped region is defined as a square region with twice the width and height of the target element, extending half of the element's width and height in all directions. When the expanded region exceeds the image boundaries, the crop is clipped accordingly. RelationQA thus provides a focused evaluation of agents' fine-grained functional understanding in GUI environments

4. Method: RelationAgent

Leveraging data from RelationGUI, we introduce RelationAgent, an exploratory framework that augments MLLMs with the ability to understand spatial, functional, and hierarchical relationships among GUI elements. The framework adopts a progressive three-stage fine-tuning paradigm, where each stage targeting a specific competency (see Fig. 5). First, the model is trained on grounding tasks to localize target elements from natural-language instructions. Next, relational supervision from RelationGUI is incorporated to enable the model to capture structured inter-element relationships. Finally, the model is fine-tuned on multi-step navigation tasks that emulate real-world user interactions. Our fine-tuning is based on the Supervised Fine-Tuning (SFT):

$$\mathcal{L}_{\text{SFT}}(\pi_{\theta}) = -\mathbb{E}_{(x,y) \sim D} [\log \pi_{\theta}(y | x)] \quad (1)$$

where x is the input, y denotes the ground-truth output corresponding to input x .

4.1. Stage 1: Grounding training

As the first step and core foundation of RelationAgent, the grounding stage is designed to accurately localize the target GUI element referenced in a user instruction Q , thereby providing the basis for subsequent relational understanding and multi-step navigation. Formally, the agent predicts the bounding box $b(e_i) = (x_i, y_i, w_i, h_i)$ of the target element e_i :

$$\text{Grounding}(I, Q, E) \rightarrow P(b(e_i) | I, Q, E) \quad (2)$$

where I denotes the GUI image, Q is the natural language instruction, and E represents the set of GUI elements in I (which can be omitted [5]).

In this stage, the model is trained to localize GUI elements referenced in natural language instructions, learning to predict precise bounding boxes for queried interface components. It is worth noting that accurate grounding is not only about identifying a single element, but also provides a starting point for subsequent analysis of its potential relationships with other elements spatially and visually.

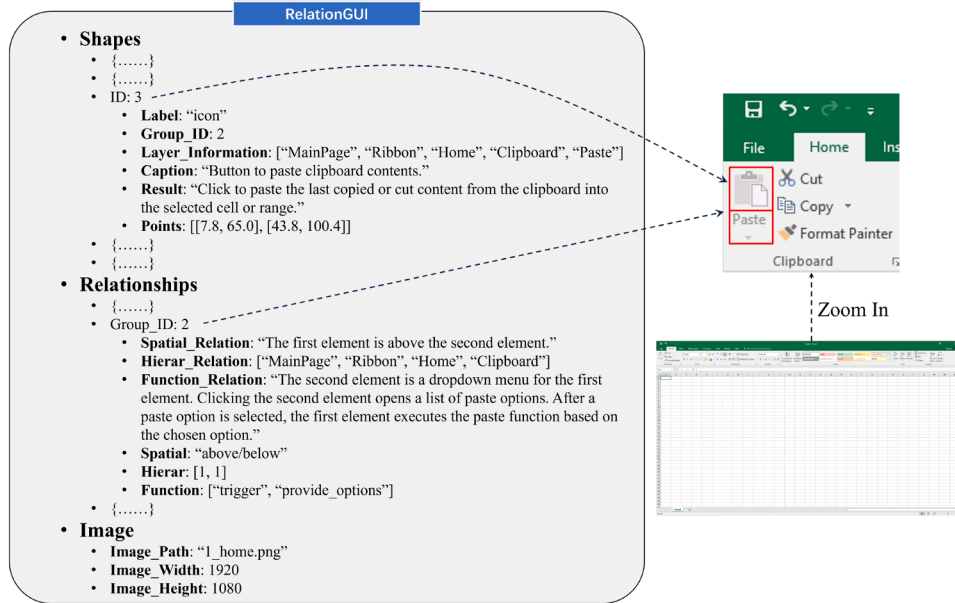


Fig. 3. An annotation example in RelationGUI. The left side presents the annotated JSON information, while the right side shows the corresponding GUI image.

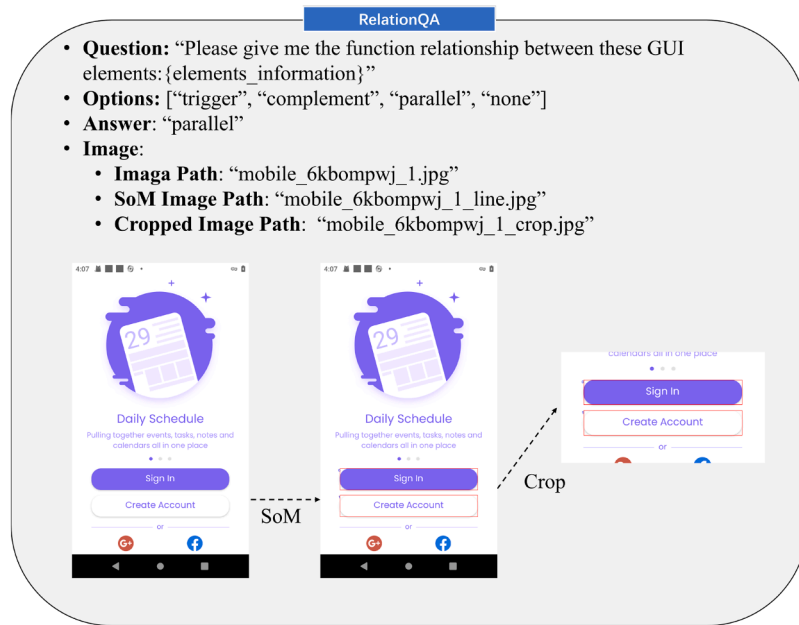


Fig. 4. An example in RelationQA benchmark. The top side displays the benchmark content, while the bottom side presents the corresponding GUI images, including the original image, the image after SoM (Set-of-Mark) processing, and the cropped image.

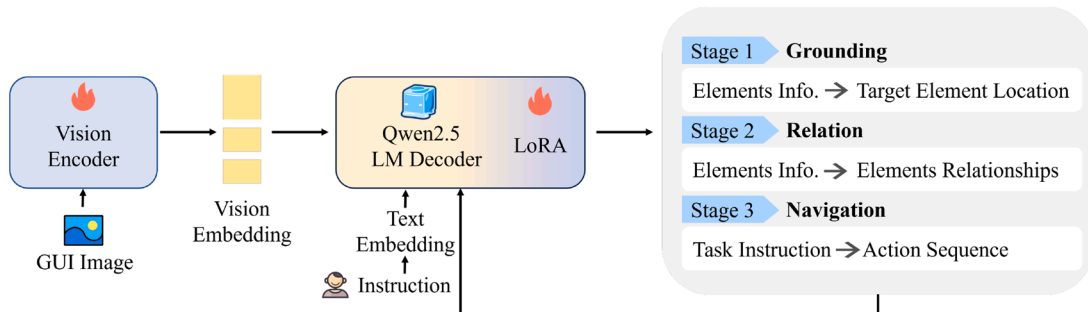


Fig. 5. An overview of our fine-tuning architecture with a focus on the three-stage training paradigm: Grounding, Relation, and Navigation.

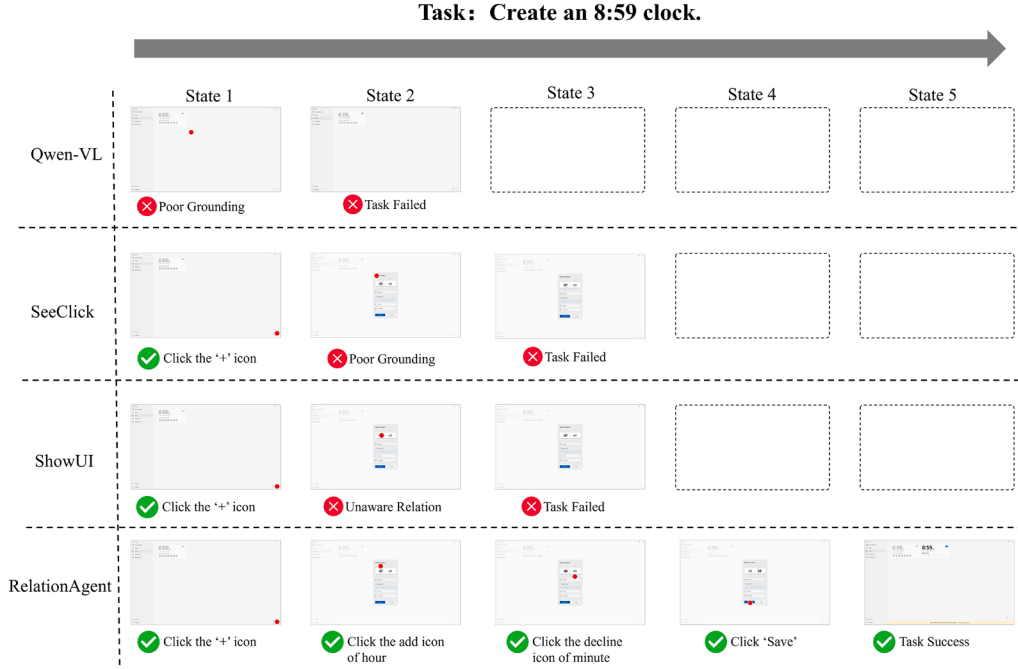


Fig. 6. Case study of clock task on Windows. Each picture represents the current clock state, and the red dot represents the click position predicted by the GUI agent. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)



Fig. 7. Case studies on RelationQA. Accurately described answer is marked in green, while inaccurately described ones in red. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

4.2. Stage 2: Relation training

Following the GUI element grounding capabilities established in the first phase, the second stage aims to explicitly identify and model the relationships among GUI elements, which is an essential component for enabling contextual understanding in GUI agents. Each relation is rep-

resented as a triplet:

$$r = (e_i, \rho_{ij}, e_j) \quad (3)$$

where e_i and e_j are GUI elements and ρ_{ij} denotes the specific relation between them. The overall relation set $\mathcal{R}$ in RelationGUI is defined as:

$$\mathcal{R} = \{(e_i, \rho_{ij}, e_j) \mid e_i, e_j \in E, \rho_{ij} \in \{\rho_{ij}^{spatial}, \rho_{ij}^{func}, \rho_{ij}^{hierar}\}\} \quad (4)$$

Table 2

Zero-shot grounding on ScreenSpot [43]. Size refers to model size. The best score for each metric is highlighted in bold, with the second-best score underlined.

Method	Size	Mobile		Desktop		Web		Avg.
		Text	Icon	Text	Icon	Text	Icon	
Qwen-VL [18]	7B	9.0	5.7	8.2	4.3	1.7	6.4	6.1
Qwen2-VL [4]	7B	52.8	46.4	47.9	29.3	29.1	26.1	39.8
Qwen2.5-VL [41]	3B	62.1	46.4	54.1	30.0	31.2	48.3	46.9
Qwen2.5-VL [41]	7B	<u>93.4</u>	76.4	<u>87.6</u>	<u>57.9</u>	82.2	63.1	<u>78.6</u>
CogAgent [42]	18B	67.0	24.0	74.2	20.0	70.4	28.6	47.4
SeeClick [43]	9B	78.0	52.0	72.2	30.0	55.7	32.5	53.4
UGround [9]	7B	82.8	60.3	82.5	63.6	80.4	70.4	73.3
ShowUI [5]	2B	91.6	69.0	81.8	59.0	<u>83.0</u>	65.5	74.9
RelationAgent	3B	95.2	<u>74.7</u>	88.7	54.3	83.0	<u>67.0</u>	79.2

The objective of this phase is to train the model to predict relation triplets $\mathcal{R}$ from the element set E and their visual representations. Supervision is provided using fine-grained relational annotations from *RelationGUI*, enabling the model to learn both inter-element dependencies and relation semantics. To explicitly integrate relational understanding into the model, we translate each relation triplet into a natural language description. We use these descriptions to fine-tune MLLMs, guiding the model to associate structured relational semantics with visual context. This strategy encourages the model not only to identify GUI components but also to understand how elements interact within interface layouts. In this way, the model not only learns to recognize individual elements, but more importantly, it understands how these elements are related to each other, laying the foundation for more precise navigation and task planning.

4.3. Stage 3: Navigation training

Navigation refers to the agent's capability to accomplish complex GUI-based tasks by interpreting user instructions into a sequence of executable actions. The output action sequence is denoted as $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$, where each action a_i is defined as:

$$a_i = (\text{type}_i, b_i, \text{args}_i) \quad (5)$$

with type_i specifies the action type (e.g., `click`, `input`), b_i indicates the target element's bounding box, and args_i represents optional arguments such as input text. Therefore, the navigation task is then formulated as a mapping from the multi-modal inputs to the action sequence:

$$\text{Navigation} : (I, Q, E) \rightarrow \mathcal{A} \quad (6)$$

To enhance contextual understanding, relation information $\mathcal{R}$ can be incorporated, leading to an extended formulation:

$$\text{Navigation} : (I, Q, E, \mathcal{R}) \rightarrow \mathcal{A} \quad (7)$$

In the final stage, the model is optimized for end-to-end task execution. Given a high-level instruction, it predicts and executes a sequence of GUI actions. By leveraging the relational representations acquired in Stage 2, the model can reason over inter-element dependencies, leading to more accurate and robust decision-making in complex, multi-step interaction scenarios.

5. Experiments

5.1. Experiments setup

5.1.1. Training dataset

To train RelationAgent across its three-stage pipeline, we adopt stage-specific datasets aligned with their respective learning objectives. For the grounding stage, we utilize GUI element annotations from our proposed RelationGUI, supplemented with filtered samples from the UGround dataset [9] to increase grounding diversity and coverage.

Table 3

Experimental results on RelationQA, categorized by platform.

Method	Size	Mobile	Desktop	Web	Avg.
Qwen2-VL [4]	2B	2.5	25.4	7.2	25.3
Qwen2.5-VL [41]	3B	<u>47.5</u>	5.1	52.7	24.0
Qwen2.5-VL [41]	7B	<u>52.0</u>	12.1	<u>54.1</u>	29.2
UGround [9]	7B	36.5	24.4	15.3	24.8
ShowUI [5]	2B	15.5	43.6	9.9	<u>30.6</u>
RelationAgent	3B	61.0	<u>41.2</u>	61.7	49.7

Table 4

Experimental results on RelationQA, categorized by different functional relationships type.

Method	Size	Trigger	Complement	Parallel	None	Avg.
Qwen2-VL [4]	2B	49.1	4.6	0.4	10.0	16.8
Qwen2.5-VL [41]	3B	6.0	2.7	94.0	1.2	24.0
Qwen2.5-VL [41]	7B	7.4	24.0	<u>92.6</u>	0.0	29.2
UGround [9]	7B	<u>44.2</u>	8.9	38.7	7.6	24.8
ShowUI [5]	2B	33.5	72.3	0.0	<u>12.4</u>	<u>30.6</u>
RelationAgent	3B	41.6	<u>48.1</u>	58.3	49.2	49.7

For the relation stage, we rely on the relational annotations (spatial, functional, and hierarchical relationships) provided in RelationGUI, enabling the model to learn structured inter-element dependencies. For the navigation stage, we employ the Mind2Web dataset [12], which supports instruction-following agents by providing multi-step, goal-oriented tasks across a wide variety of real-world websites.

5.1.2. Benchmark

- **Grounding.** We adopt the commonly used ScreenSpot benchmark [43] to evaluate the model's grounding performance. ScreenSpot is a zero-shot GUI grounding benchmark that covers diverse interfaces across desktop, mobile, and web platforms. It provides separate evaluation for textual and widget-level localization, making it well-suited for assessing fine-grained grounding accuracy.
- **Relation.** To assess relational reasoning over GUI elements, we evaluate models on the proposed RelationQA benchmark, which consists of single-choice questions targeting functional relationships. Performance on RelationQA is measured using top-1 exact-match accuracy: a prediction is deemed correct only if the selected answer exactly matches the ground-truth label. As RelationQA is formulated as a single-label multiple-choice task rather than a detection problem, precision and recall metrics are not applicable and thus omitted. To prevent data leakage, all RelationQA samples are strictly excluded from the relation training split of RelationGUI.
- **Navigation.** For instruction-following evaluation, we adopt Mind2Web [12], the first dataset for developing and evaluating generalist agents that can follow language instructions to complete complex tasks on any website. This benchmark remains challenging, as state-of-the-art models achieving only modest performance, underscoring its suitability for evaluating complex reasoning and planning abilities in real-world web environments.

5.1.3. Training details

We use Qwen2.5-VL-3B [41] as base model for the three-stage training. For the grounding and relation stage, we set learning rates at $1e-4$. LoRA [44] was applied to improve training efficiency, with rank and alpha values of 32 and 16. Grounding and relation stage involved training for 1 epoch using the AdamW optimizer. The Navigation stage was applied LoRA with rank and alpha values of 128 and 256, learning rate of $2e-5$, and trained for 2 epochs. The batch size was set to 8 for three stage.

Table 5
Performance of screen navigation on Mind2Web.

Method	Size	Cross-Task			Cross-Website			Cross-Domain		
		Ele.Acc	Op.F1	Step.SR	Ele.Acc	Op.F1	Step.SR	Ele.Acc	Op.F1	Step.SR
Qwen2.5-VL	3B	2.5	14.5	0.4	2.7	12.6	1.0	3.3	24.2	1.7
Qwen2.5-VL	7B	6.2	72.8	5.2	6.3	68.2	4.5	8.4	73.6	7.2
CogAgent	18B	22.4	53.0	17.6	18.4	42.4	13.4	20.6	42.0	15.5
SeeClick	9B	28.3	87.0	25.5	21.4	80.6	16.4	23.2	84.8	20.8
ShowUI	2B	<u>39.7</u>	<u>88.0</u>	<u>36.9</u>	<u>41.0</u>	<u>83.6</u>	<u>34.2</u>	<u>38.9</u>	<u>85.3</u>	<u>34.1</u>
RelationAgent	3B	43.2	87.3	39.7	41.3	84.8	36.5	39.4	86.1	41.5

5.2. Main results

We first evaluate the performance of RelationAgent on grounding, relation, and navigation benchmark. Then, we analyze the impact of relation-stage training on the grounding and navigation abilities of the RelationAgent. Finally, we present several qualitative analyses to further illustrate the importance of the relation-stage training.

5.2.1. Grounding performance on ScreenSpot

In Table 2, we present zero-shot grounding results on the ScreenSpot. From the result, we observe that (1) Across all methods, the text-track scores are consistently higher than the icon-track scores, which means the icon track is more challenging due to its visual grounding. (2) Mobile scores are significantly higher than desktop and web, highlighting the lack of visual GUI grounding data beyond mobile devices. (3) Our proposed method RelationAgent demonstrates strong grounding capabilities and outperforms most baselines in both textual and icon tracks. However, its performance on ScreenSpot remains limited, which we attribute to the scarcity of relational grounding tasks in the benchmark. Since the benchmark primarily evaluates direct grounding without requiring intermediate relational reasoning, the benefits introduced by our relation modeling stage are not fully reflected in this evaluation.

5.2.2. Relation performance on RelationQA

Table 3 presents the results on the RelationQA benchmark. Overall, existing methods exhibit limited relational reasoning capability, with accuracy scores falling below 30%. This suggests that current models struggle to interpret functional relationships among elements when faced with structured relational questions. We hypothesize two primary contributing factors: (1) Relational reasoning ability is inherently grounded in accurate element localization. Without reliable grounding, agents fail to correctly identify the elements referenced in the question, making it impossible to infer their relationships. (2) The lack of explicit relational supervision during training leads to poor generalization on relational tasks.

In Table 4, we further report the accuracy of different agents across four types of functional relationships. We observe that, except for RelationAgent, most models exhibit a strong bias toward a single relationship type. For instance, Qwen2.5-VL predominantly selects *Parallel*, while ShowUI heavily favors *Complement*. This bias in prediction distribution indicates that these models fail to grasp and understand the diversity of relationships, leading to judgment failures. In contrast, RelationAgent demonstrates a more balanced prediction distribution across four categories, indicating a better understanding of diverse functional relationships. Although its accuracy on certain sub-types is not consistently the highest, this outcome highlights the inherent difficulty of the benchmark.

Collectively, the results in Tables 3 and 4 highlight the effectiveness of relation-stage training. RelationAgent consistently outperforms all baseline models on RelationQA, demonstrating that explicit relational modeling leads to substantial improvements in relational reasoning for GUI agents.

Table 6

Ablation study on ScreenSpot, highlighting the effect of the relation stage on grounding performance.

Method	Mobile		Desktop		Web		Avg.
	Text	Icon	Text	Icon	Text	Icon	
RelationAgent	95.2	74.7	88.7	54.3	83.0	67.0	79.2
w/o Relation	<u>94.9</u>	<u>72.1</u>	<u>88.7</u>	53.6	<u>82.2</u>	63.6	<u>77.9</u>
w/o Navigation	93.4	72.0	87.6	54.3	82.2	63.1	77.4
w/o Relation + Navigation	92.3	71.2	87.1	51.4	81.7	<u>64.1</u>	76.7

Table 7

Ablation study on ScreenSpot, highlighting the effect of different type of relation on grounding performance.

Method	Mobile		Desktop		Web		Avg.
	Text	Icon	Text	Icon	Text	Icon	
RelationAgent	95.2	74.7	88.7	54.3	83.0	67.0	79.2
w/o Relation	94.9	72.1	88.7	53.6	82.2	63.6	77.9
w/o spatial	<u>95.2</u>	<u>74.2</u>	<u>88.7</u>	54.3	<u>82.6</u>	<u>66.5</u>	<u>79.0</u>
w/o functional	94.9	72.5	88.7	53.6	82.2	64.1	78.1
w/o hierarchical	95.2	73.8	88.7	54.3	82.6	65.5	78.9

5.2.3. Navigation performance on Mind2Web

Table 5 reports the performance of RelationAgent on the Mind2Web benchmark. Overall, RelationAgent achieves competitive results across multiple evaluation metrics and consistently outperforms strong baselines, indicating its ability to generalize to complex web-based navigation tasks. While its performance is lower on certain metrics, we attribute this to the nature of the Mind2Web benchmark, which contains relatively few tasks that explicitly require relational understanding. As a result, the benefits of our relation-aware training may not be fully captured in this benchmark setting.

5.3. Ablation study

To analyze the contribution of the relation stage, we conduct an ablation study by removing relation-stage training from RelationAgent and evaluating the resulting impact on grounding and navigation performance.

5.3.1. Grounding

5.3.1.1. Relation stage. Table 6 presents the ablation results on ScreenSpot with and without the relation-stage training. The results show that removing the relation stage leads to a minor performance drop. This observation is consistent with our expectations, as the ScreenSpot benchmark is primarily designed to evaluate direct grounding and does not explicitly target relational reasoning. Nonetheless, the slight performance drop suggests that relational information can still offer auxiliary cues, potentially supporting grounding performance even when relational reasoning is not the primary focus.

5.3.1.2. Relation type. To better understand the specific impact of each type of relation, we perform a detailed ablation study on spatial, functional, and hierarchical relationships using the ScreenSpot benchmark.

Table 8

Ablation study on Mind2Web, highlighting the effect of the relation stage on navigation performance.

Method	Cross-Task			Cross-Website			Cross-Domain		
	Ele.Acc	Op.F1	Step.SR	Ele.Acc	Op.F1	Step.SR	Ele.Acc	Op.F1	Step.SR
RelationAgent	43.2	87.3	39.7	41.3	84.8	36.5	39.4	86.1	41.5
w/o Relation	42.4(↓)	86.4(↓)	37.2(↓)	41.6(↑)	83.5(↓)	35.5(↓)	38.5(↓)	84.8(↓)	35.2(↓)

As shown in Table 7, removing functional relationships results in the largest drop in performance, indicating that functional relationships play a key role in relational understanding. In contrast, removing spatial and hierarchical relationships leads to smaller performance decreases. These results support our hypothesis that functional relationships are essential for reasoning about graphical user interfaces, while spatial and hierarchical relationships mainly offer structural context.

5.3.2. Navigation

As shown in Table 8, removing the relation-stage training leads to a performance decline across multiple metrics on the Mind2Web benchmark. Consistent with our analysis of ScreenSpot, the limited presence of relational tasks results in only a modest drop. Nevertheless, this reduction is sufficient to support our claim that explicit relational training contributes to improved contextual understanding and task execution in GUI-based agents.

5.4. Qualitative analysis

5.4.1. RelationGUI cases

Fig. 6 presents a case study of a clock-related task from the RelationGUI dataset, captured from a real GUI environment on Windows. As shown in the figure, only RelationAgent successfully completes the task. Qwen-VL and SeeClick fail due to limited grounding ability, while ShowUI fails as it lacks relational understanding. In contrast, RelationAgent achieves success by leveraging both strong grounding and relational understanding ability.

5.4.2. RelationQA cases

Fig. 7 presents several representative examples from the RelationQA benchmark, highlighting the comparative performance of different GUI agents in identifying functional relationships among interface elements. Across all four cases, RelationAgent consistently provides accurate predictions, demonstrating a more robust understanding of relational dependencies. In contrast, other methods frequently misclassify relationships, indicating limitations in relational understanding. These results underscore the effectiveness of explicit relation modeling in enhancing the interpretability and reasoning capabilities of GUI agents.

6. Discussion

6.1. Application scenarios

The proposed RelationGUI dataset and the RelationAgent framework open up a range of potential applications in GUI understanding and automation. First, relational reasoning among GUI elements can enhance the robustness of GUI grounding and navigation models, particularly in tasks that require relational understanding and contextual or dependency-based decisions. Second, the relational annotations in RelationGUI can support the development of visual reasoning models for accessibility tools, helping systems better interpret interface layouts for visually impaired users. Third, RelationGUI provides a structured foundation for studying layout understanding and functional pattern recognition, which are relevant to human-computer interaction research. Finally, the relational modeling framework introduced in this work can also contribute to multi-modal reasoning systems that integrate vision, text, and interaction history for more general-purpose interface agents.

6.2. Limitations

Although RelationGUI and RelationAgent represent a solid step toward explicit relational understanding in GUI agents, several limitations remain.

- The current taxonomy of inter-element relationships is limited to three major categories-positional, functional, and hierarchical. Functional relationships are further divided into four subtypes: trigger, complement, parallel, and none. In future work, we plan to explore more fine-grained or dynamic relationship representations that better capture semantic dependencies and multi-step interactions within complex interfaces.
- The dataset currently contains 37,724 annotated instances. While this scale is sufficient for initial analysis, larger and more diverse datasets are needed to improve model generalization across platforms and interaction paradigms.
- Each sample in RelationGUI consists of a single static image with grounding and relational annotations. However, the absence of execution traces limits the dataset's ability to model temporal dependencies and user interaction dynamics. Incorporating sequence-level data will be crucial for tasks that involve reasoning over action chains or interface state changes.
- The evaluation of RelationAgent relies mainly on existing grounding and navigation benchmarks, which contain relatively few relational queries. As a result, they provide only a partial assessment of relational reasoning ability. Further benchmarks specifically designed to emphasize relational inference are necessary for a more comprehensive evaluation.

6.3. Future work

Future extensions of this work will proceed along several directions. First, We plan to incorporate more precise and diverse annotation types in RelationGUI to enhance the dataset's usability and scalability. Second, we aim to integrate other relational representation into RelationAgent to better model structured dependencies and improve generalization to unseen layouts. Third, we will explore the adaptation of RelationAgent to real-world applications, enabling a broader evaluation of its practical utility. Finally, we plan to release additional relational benchmarks to facilitate systematic comparison across models and to promote progress in relational reasoning for GUI agents.

7. Conclusion

In this work, we address the limited treatment of relational understanding in existing GUI agents by introducing RelationGUI, a large-scale, cross-platform dataset with explicit and fine-grained relational annotations among GUI elements. By systematically covering spatial, hierarchical, and functional relations across mobile, desktop, and web interfaces, RelationGUI provides a structured dataset and benchmark for analyzing relational reasoning beyond isolated grounding accuracy. Building on this dataset, we propose RelationAgent, a three-stage framework that explicitly incorporates relational reasoning as an intermediate learning stage. Experimental results demonstrate that relation-stage supervision not only improves performance on relational reasoning benchmarks, but also yields consistent gains in grounding and navigation

tasks. These findings highlight the role of relational modeling as a unifying factor that enhances multiple aspects of GUI agent, rather than serving as an isolated capability.

Despite these strengths, our work has several limitations. First, relational annotations in RelationGUI are derived from predefined relation categories, which may not fully capture more complex or implicit relationships present in real-world interfaces. Second, RelationAgent follows a staged training paradigm that introduces additional design complexity and training cost compared to end-to-end approaches. Moreover, while our evaluation spans multiple platforms, the current benchmarks primarily focus on static screenshots and do not fully account for dynamic interface changes or long-horizon interaction dependencies.

Nevertheless, we believe that RelationGUI, RelationQA and RelationAgent offer valuable resources for the community. The dataset and benchmark establish a common ground for evaluating relational reasoning in GUI environments, while the agent framework provides a reference design for integrating relational supervision into GUI agent pipelines. Researchers can leverage RelationQA to analyze failure modes related to relational ambiguity, dataset bias, or cross-platform generalization, and adapt the proposed framework to alternative architectures or learning paradigms.

Future work can extend this direction in several ways. One promising avenue is to incorporate dynamic and temporal relations arising from state transitions and user interactions. Another direction is to explore more flexible or open-vocabulary relational representations that reduce reliance on predefined relation taxonomies. Finally, integrating relational reasoning with MLLMs in a training-efficient or training-free manner remains an open challenge, and we hope that this work motivates further research toward more robust and generalizable GUI agents with deeper relational understanding.

CRedit authorship contribution statement

Longhui Ma: Writing – original draft, Software, Resources, Project administration; **Di Zhao:** Conceptualization; **Siwei Wang:** Writing – review & editing, Resources, Methodology; **Zhao Lv:** Methodology; **Miao Wang:** Writing – review & editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by the [National Natural Science Foundation of China](#) (project nos. 62406329, 62476280).

Supplementary material

Supplementary material associated with this article can be found in the online version at [10.1016/j.patcog.2026.113262](https://doi.org/10.1016/j.patcog.2026.113262).

References

- [1] B. Shneiderman, C. Plaisant, *Designing the user interface: strategies for effective human-computer interaction*, Pearson Education India, 2010.
- [2] S. Xiao, Y. Chen, Y. Song, L. Chen, L. Sun, Y. Zhen, Y. Chang, T. Zhou, UI semantic component group detection: grouping UI elements with similar semantics in mobile graphical user interface, *Displays* 83 (2024) 102679.
- [3] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F.L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat, et al., Gpt-4 technical report, [arXiv:2303.08774](https://arxiv.org/abs/2303.08774) (2023).

- [4] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, et al., Qwen2-vl: enhancing vision-language model's perception of the world at any resolution, [arXiv:2409.12191](https://arxiv.org/abs/2409.12191) (2024).
- [5] K.Q. Lin, L. Li, D. Gao, Z. Yang, S. Wu, Z. Bai, S.W. Lei, L. Wang, M.Z. Shou, Showui: one vision-language-action model for gui visual agent, in: *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 19498–19508.
- [6] Y. Yang, Y. Wang, D. Li, Z. Luo, B. Chen, C. Huang, J. Li, Aria-ui: visual grounding for gui instructions, in: *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 22418–22433.
- [7] Y. Sun, S. Zhao, T. Yu, H. Wen, S. Va, M. Xu, Y. Li, C. Zhang, Gui-xplore: empowering generalizable gui agents with one exploration, in: *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 19477–19486.
- [8] Q. Wu, K. Cheng, R. Yang, C. Zhang, J. Yang, H. Jiang, J. Mu, B. Peng, B. Qiao, R. Tan, et al., GUI-Actor: coordinate-free visual grounding for GUI agents, [arXiv:2506.03143](https://arxiv.org/abs/2506.03143) (2025).
- [9] B. Gou, R. Wang, B. Zheng, Y. Xie, C. Chang, Y. Shu, H. Sun, Y. Su, Navigating the digital world as humans do: universal visual grounding for GUI agents, in: *The Thirteenth International Conference on Learning Representations*, 2025.
- [10] Z. Wu, Z. Wu, F. Xu, Y. Wang, Q. Sun, C. Jia, K. Cheng, Z. Ding, L. Chen, P.P. Liang, et al., OS-ATLAS: foundation action model for generalist GUI agents, in: *The Thirteenth International Conference on Learning Representations*, 2025.
- [11] W. Li, W.E. Bishop, A. Li, C. Rawles, F. Campbell-Ajala, D. Tyamagundlu, O. Riva, On the effects of data scale on ui control agents, *Adv. Neural Inf. Process. Syst.* 37 (2024) 92130–92154.
- [12] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, Y. Su, Mind2web: towards a generalist agent for the web, *Adv. Neural Inf. Process. Syst.* 36 (2023) 28091–28114.
- [13] Y. Lu, J. Yang, Y. Shen, A. Awadallah, Omniparser for pure vision based gui agent, [arXiv:2408.00203](https://arxiv.org/abs/2408.00203) (2024).
- [14] Z. Wang, W. Chen, L. Yang, S. Zhou, S. Zhao, H. Zhan, J. Jin, L. Li, Z. Shao, J. Bu, Mp-gui: modality perception with mllms for gui understanding, in: *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 29711–29721.
- [15] Y. Liu, P. Li, Z. Wei, C. Xie, X. Hu, X. Xu, S. Zhang, X. Han, H. Yang, F. Wu, InfiGUI-Agent: a multimodal generalist GUI agent with native reasoning and reflection, in: *ICML 2025 Workshop on Computer Use Agents*.
- [16] A. Hurst, A. Lerer, A.P. Goucher, A. Perelman, A. Ramesh, A. Clark, A.J. Ostrow, A. Welihinda, A. Hayes, A. Radford, et al., Gpt-4o system card, [arXiv:2410.21276](https://arxiv.org/abs/2410.21276) (2024).
- [17] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A.M. Dai, A. Hauth, K. Millican, et al., Gemini: a family of highly capable multimodal models, [arXiv:2312.11805](https://arxiv.org/abs/2312.11805) (2023).
- [18] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, J. Zhou, Qwen-VL: a versatile vision-language model for understanding, localization, text reading, and beyond, [arXiv:2308.12966](https://arxiv.org/abs/2308.12966) (2023).
- [19] K. Lee, Towards a working definition of designing generative user interfaces, in: *Companion Publication of the 2025 ACM Designing Interactive Systems Conference*, 2025, pp. 489–495.
- [20] J. Wang, H. Xu, J. Ye, M. Yan, W. Shen, J. Zhang, F. Huang, J. Sang, Mobile-agent: autonomous multi-modal mobile device agent with visual perception, in: *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- [21] Y. Xu, Z. Wang, J. Wang, D. Lu, T. Xie, A. Saha, D. Sahoo, T. Yu, C. Xiong, Aguis: unified pure vision agents for autonomous gui interaction, [arXiv:2412.04454](https://arxiv.org/abs/2412.04454) (2024).
- [22] A.K. Jain, R.P.W. Duin, J. Mao, Statistical pattern recognition: a review, *IEEE Trans. Pattern Anal. Mach. Intell.* 22 (1) (2000) 4–37.
- [23] M. Paolanti, E. Frontoni, Multidisciplinary pattern recognition applications: a review, *Comput. Sci. Rev.* 37 (2020) 100276.
- [24] W. Ju, Z. Mao, Z. Qiao, Y. Qin, S. Yi, Z. Xiao, X. Luo, Y. Fu, M. Zhang, Focus on informative graphs! semi-supervised active learning for graph-level classification, *Pattern Recognit.* 153 (2024) 110567.
- [25] Y. Jiang, C. Zhou, V. Garg, A. Oulasvirta, Graph4gui: graph neural networks for representing graphical user interfaces, in: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–18.
- [26] J. Fu, X. Zhang, Y. Wang, W. Zeng, N. Zheng, Understanding mobile GUI: from pixel-words to screen-sentences, *Neurocomputing* 601 (2024) 128200.
- [27] Y. Qian, Y. Lu, A.G. Hauptmann, O. Riva, Visual grounding for user interfaces, in: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, 2024, pp. 97–107.
- [28] E.Z. Liu, K. Guu, P. Pasupat, T. Shi, P. Liang, Reinforcement learning on web interfaces using workflow-guided exploration, in: *International Conference on Learning Representations*, 2018.
- [29] S. Yao, H. Chen, J. Yang, K. Narasimhan, Webshop: towards scalable real-world web interaction with grounded language agents, *Adv. Neural Inf. Process. Syst.* 35 (2022) 20744–20757.
- [30] C. Rawles, A. Li, D. Rodriguez, O. Riva, T. Lillicrap, Androidinthewild: a large-scale dataset for android device control, *Adv. Neural Inf. Process. Syst.* 36 (2023) 59708–59728.
- [31] R. Kapoor, Y.P. Butala, M. Russak, J.Y. Koh, K. Kamble, W. AlShikh, R. Salakhutdinov, Omniact: a dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web, in: *European Conference on Computer Vision*, 2024, pp. 161–178.
- [32] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T.J. Hua, Z. Cheng, D. Shin, F. Lei, et al., Oworld: benchmarking multimodal agents for open-ended tasks in real computer environments, *Adv. Neural Inf. Process. Syst.* 37 (2024) 52040–52094.

- [33] D. Chen, Y. Huang, S. Wu, J. Tang, H. Zhou, Q. Zhang, Z. He, Y. Bai, C. Gao, L. Chen, et al., GUI-world: A video benchmark and dataset for multimodal GUI-oriented understanding, in: *The Thirteenth International Conference on Learning Representations*, 2025.
- [34] B. Deka, Z. Huang, C. Franzen, J. Hibschan, D. Afargan, Y. Li, J. Nichols, R. Kumar, Rico: a mobile app dataset for building data-driven design applications, in: *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, 2017, pp. 845–854.
- [35] Q. Lu, W. Shao, Z. Liu, L. Du, F. Meng, B. Li, B. Chen, S. Huang, K. Zhang, P. Luo, GUIOdyssey: a comprehensive dataset for cross-app GUI navigation on mobile devices, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 22404–22414.
- [36] A. Norman Donald, *The design of everyday things*, MIT Press, 2013.
- [37] M. Clement, C. Kurtz, L. Wendling, Learning spatial relations and shapes for structural object description and scene recognition, *Pattern Recognit.* 84 (2018) 197–210.
- [38] D. Shi, W. Cui, D. Huang, H. Zhang, N. Cao, Reverse-engineering information presentations: recovering hierarchical grouping from layouts of visual elements, *Visual Intell.* 1 (1) (2023) 9.
- [39] L. Gao, L. Zhang, S. Wang, Y. Li, M. Xu, Mobileviews: a large-scale mobile gui dataset, *arXiv:2409.14337* (2024).
- [40] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, J. Gao, Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v, *arXiv:2310.11441* (2023).
- [41] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, H. Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, P. Wang, W. Ding, Z. Fu, Y. Xu, J. Ye, X. Zhang, T. Xie, Z. Cheng, H. Zhang, Z. Yang, H. Xu, J. Lin, Qwen2.5-VL Technical Report, *arXiv:2502.13923* (2025).
- [42] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding, et al., Cogagent: a visual language model for gui agents, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 14281–14290.
- [43] K. Cheng, Q. Sun, Y. Chu, F. Xu, L. YanTao, J. Zhang, Z. Wu, SeeClick: harnessing gui grounding for advanced visual gui agents, in: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 9313–9332.
- [44] E.J. Hu, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al., LoRA: low-rank adaptation of large language models, in: *International Conference on Learning Representations*, 2022.